FixFast on AWS — Architecture Write-up

Oct 14, 2026 · @Abhishek Sharma

1. Executive summary

Run FixFast as two small containerised Node.js services on **Amazon ECS with AWS Fargate** behind an Application Load Balancer, with the React site on **S3 + CloudFront**, data in **RDS for PostgreSQL**, photos in a private **S3** bucket, jobs on **Amazon SQS**, and sign-in through **Amazon Cognito federated with Google**. Everything lives in one Region (assumed ap-south-1, Mumbai) across two Availability Zones.

The design keeps the booking request fast and safe: the API saves the booking and a matching job row in one database transaction, replies immediately, and a background worker sends the email (Amazon SES) and SMS (AWS End User Messaging SMS) with retries and a dead-letter queue. Customers can never reach staff screens because the API, not the browser, checks the `staff` group in every token.

Dev and staging run one task per service, one NAT gateway and a Single-AZ database; production adds Multi-AZ RDS, a second NAT gateway and WAF. Every resource is defined in AWS CDK and deployed from GitHub Actions.

2. Architecture overview

The system has three layers: a public edge (Route 53, CloudFront, WAF, Cognito), a private application tier inside a VPC (ALB, API and worker containers, PostgreSQL), and regional managed services the app calls (S3, SQS, SES, SMS, Secrets Manager, CloudWatch). The draw.io diagram shows these as nested boundaries with numbered flows 1–12.

Requirements stated in the exercise

Requirement	How it is met
Customers book a repair	React form → `POST /api/bookings` → RDS PostgreSQL
Customers upload an appliance photo	Presigned S3 upload straight from the browser
Staff see and assign bookings	Admin screens + `/api/admin/*` routes, allowed only for the Cognito `staff` group

Requirement	How it is met
Confirmation SMS and email sent in the background	SQS queue + ECS worker → SES and End User Messaging SMS
Website never waits for notifications	API returns 201 after the DB commit; sending happens asynchronously
Google sign-in for customers and staff	Cognito user pool with Google as a federated identity provider
Cover networking, compute, storage, auth, config/secrets, monitoring, deployment	Sections 3–8

Assumptions introduced (none are stated in the exercise)

- Customers are mainly in India, so the Region is **ap-south-1 (Mumbai)**. This also matters for SMS: local Indian routes need DLT registration and are only supported from Mumbai or Hyderabad (AWS docs).
- Low traffic: hundreds of bookings a day, well under 10,000 monthly active users.
- Photos up to 10 MB, JPEG/PNG/HEIC only.
- Target availability around 99.9% with RPO ⩽ 5 minutes and RTO under 1 hour; a single Region is acceptable.
- A small team that prefers managed services and writes TypeScript, so AWS CDK fits.

Main failure scenarios considered: loss of one Availability Zone, a crashed container, a bad deployment, a database failover, SES or SMS being unavailable or rejecting a message, a worker crashing mid-job, and a malicious user editing the frontend.

3. AWS services and their responsibilities

Each of the seven parts maps to one primary AWS service; supporting services are only added where they remove real operational work.

Area	Service	What it does here	Why this one	Connects to
DNS	Route 53	Hosts `fixfast.com`, alias record to CloudFront	Native alias to CloudFront, health-checked DNS	CloudFront

Area	Service	What it does here	Why this one	Connects to
Frontend	S3 + CloudFront	Serves the React build; `/api/*` is forwarded to the ALB	Static hosting needs no servers; one domain means no CORS; edge TLS and caching	S3 (Origin Access Control), ALB, WAF, ACM
Edge security	AWS WAF + ACM	Managed rule groups and rate limits; free TLS certificates	Blocks common attacks before they reach the app	CloudFront, ALB
API entry	Application Load Balancer	HTTPS listener, health checks, spreads traffic over tasks in two AZs	Long-lived container targets, cheap at steady traffic	ECS API tasks
Compute	ECS on Fargate	Two services from the same repo: **api** and **worker**	No servers or clusters to patch; per-task IAM roles; easy autoscaling	ALB, RDS, S3, SQS, Secrets Manager
Database	RDS for PostgreSQL	Bookings, users, assignments, outbox, notification log	Managed backups, point-in-time restore, Multi-AZ failover	API and worker tasks only
Photos	S3 (private bucket)	Stores uploads under `bookings/{id}/`	Durable, cheap, presigned URLs keep large files off the API	Browser (presigned), API via gateway endpoint
Queue	SQS standard + DLQ	Holds `send-confirmation` jobs	Fully managed, at-least-once delivery, built-in redrive to DLQ	API (producer), worker (consumer)
Email	Amazon SES	Sends the confirmation email	Native, low cost, bounce/complaint events	Worker
SMS	AWS End User Messaging SMS	Sends the confirmation SMS with a registered sender ID	Native SMS with India DLT support	Worker

Area	Service	What it does here	Why this one	Connects to
Auth	Cognito user pool	Managed login, Google federation, JWTs, `staff` group	Handles OAuth/OIDC so the app never stores passwords	Browser, Google, API (JWKS)
Config	SSM Parameter Store	Non-secret settings (queue URL, bucket name, feature flags)	Free standard tier, versioned	ECS task definitions
Secrets	Secrets Manager + KMS	DB credentials with automatic rotation; encryption keys	RDS can manage its master secret in Secrets Manager	ECS (injected at start), RDS
Monitoring	CloudWatch, SNS, CloudTrail	Logs, metrics, alarms, on-call email, API audit	Native integration with every service above	All
Delivery	GitHub Actions, ECR, AWS CDK	Build, test, store images, provision infrastructure	Team already uses GitHub; CDK is TypeScript like the app	AWS via OIDC role

Essential vs optional. Essential for launch: everything in the table. Production enhancements that can wait: Multi-AZ RDS outside production, a second NAT gateway, interface VPC endpoints, GuardDuty (including malware scanning of uploaded photos), RDS Proxy, and CloudFront caching of API reads.

4. End-to-end request and data flow

Only steps 1–8 are synchronous; notification sending (9–11) happens after the customer already has their confirmation page. Step numbers match the diagram.

#	User action	Path	Security check	Sync?
1	Opens fixfast.com	Browser → Route 53 → CloudFront	TLS (ACM), WAF rules	Sync
2–3	Site loads	CloudFront → private S3 bucket	Origin Access Control; bucket blocks all public access	Sync

#	User action	Path	Security check	Sync?
4	Signs in with Google	Browser → Cognito managed login → Google → Cognito → browser (auth code + PKCE)	Google verifies identity; Cognito issues ID/access tokens	Sync
5–6	Calls the API	Browser → CloudFront `/api/*` → ALB → API task	`Authorization: Bearer <access token>`; ALB only accepts CloudFront; API verifies JWT	Sync
7	Submits booking form	API validates input, then one transaction inserts `booking` + `outbox` rows	Customer id taken from the token, never from the request body	Sync
8	Uploads a photo	API returns a presigned POST (5 min, ≤10 MB, image types); browser uploads to S3; API records the key	Signed URL scoped to one object key; bucket private, SSE-KMS	Sync
—	Gets a response	API returns `201 Created` with the booking id; UI shows confirmation	—	Sync
9	Job is queued	Outbox relay sends the message to SQS and marks the row sent	IAM: API role may only `SendMessage` to this queue	Async
10–11	Notifications sent	Worker long-polls SQS → SES email + SMS → records result → deletes message	IAM: worker may only send from the verified domain and sender ID	Async
12	Sending keeps failing	After 5 receives SQS moves the message to the DLQ; alarm fires	—	Async

Photo order. The booking is created first, then the photo is uploaded against that booking id, so a failed upload never loses a booking. Staff view photos through short-lived presigned GET URLs issued only to the `staff` group.

When a dependency fails

- *An API task crashes or an AZ is lost:* ALB health checks (`/healthz`) remove the target; ECS replaces the task; the second AZ keeps serving.

- *Database failover (Multi-AZ):* RDS promotes the standby, usually in one to two minutes; the API retries the connection with backoff and returns 503 meanwhile, so the frontend can show a retry message.
- *SQS unreachable after commit:* nothing is lost — the outbox row stays unsent and the relay retries it.
- *SES or SMS errors or throttling:* the worker does not delete the message; it reappears after the visibility timeout and is retried, up to 5 times, then goes to the DLQ.
- *Timeouts:* 5 s on DB queries, 10 s on SES/SMS calls, ALB idle timeout 60 s, so no request hangs.

5. Networking and security

Only the load balancer and NAT gateways sit in public subnets; containers and the database have no public IP and cannot be reached from the internet.

VPC and subnets. One VPC `10.0.0.0/16` (65,536 addresses, room to grow) split into three tiers in each of two AZs, all `/24`:

Tier	AZ A	AZ B	Holds	Route to internet
Public	10.0.0.0/24	10.0.1.0/24	ALB, NAT gateways	`0.0.0.0/0` → internet gateway
Private app	10.0.10.0/24	10.0.11.0/24	API and worker tasks	`0.0.0.0/0` → NAT gateway (outbound only)
Isolated DB	10.0.20.0/24	10.0.21.0/24	RDS primary and standby	None (local VPC route only)

Why each network component exists

- *Internet gateway:* lets the ALB receive traffic from CloudFront and lets NAT reach the internet.
- *NAT gateway:* the worker must call SES, End User Messaging and Cognito's public JWKS endpoint, and tasks must pull images and secrets. Trade-off: one NAT per AZ survives an AZ outage but doubles a fixed hourly charge (VPC pricing). Recommendation: one NAT in dev/staging, two in production.
- *S3 gateway endpoint:* free, keeps API-to-S3 traffic off the NAT and private. Interface endpoints (ECR, Secrets Manager, SQS, Logs) are optional; each has its own hourly charge per AZ, so add them only if NAT data charges grow or the security team wants no internet egress.

Security groups (least privilege)

Group	Inbound	Purpose
sg-alb	443 from the CloudFront origin-facing managed prefix list	Nobody can bypass CloudFront and WAF
sg-api	8080 from sg-alb	Only the ALB reaches API tasks
sg-worker	none	Worker only makes outbound calls
sg-db	5432 from sg-api and sg-worker	Only the app reaches PostgreSQL

CloudFront also adds a secret custom header that the ALB listener rule checks, closing the gap of other CloudFront distributions using the same prefix list.

Encryption. TLS everywhere in transit: ACM certificates on CloudFront and the ALB, `rds.force_ssl=1` on PostgreSQL, HTTPS to all AWS APIs. At rest: RDS, S3, SQS, Secrets Manager and CloudWatch Logs encrypted with KMS keys.

IAM. Each ECS service has its own task role, scoped to named resources:

- API: `s3:PutObject` / `GetObject` on `photos-bucket/bookings/*`, `sqs:SendMessage` on the booking queue, read its own parameters.
- Worker: `sqs:ReceiveMessage` / `DeleteMessage` on the queue, `ses:SendEmail` from the verified domain, `sms-voice:SendTextMessage`.
- Task execution role: pull from ECR, read only its own secrets, write logs.

No long-lived access keys exist anywhere; GitHub Actions uses OIDC to assume a deploy role.

Secrets. RDS manages the master password in Secrets Manager with automatic rotation; the app connects as a separate least-privilege user whose secret also rotates. ECS injects secrets as environment variables at task start, so a rotation is picked up by a rolling restart or by the app re-reading the secret on authentication failure.

Secure uploads. Presigned POST with a 5-minute expiry, a fixed object key, `content-length-range` up to 10 MB and an allowed `Content-Type`. Block Public Access is on, the bucket policy denies non-TLS requests, and a lifecycle rule removes orphaned uploads. Optional: GuardDuty Malware Protection for S3 to scan each new object.

6. Authentication and authorization

Authentication (who you are) is delegated to Google through Cognito; authorization (what you may do) is enforced by the API on every request using the `staff` group in the token and row ownership in PostgreSQL.

How sign-in works

1. The SPA redirects to the Cognito managed login with `identity_provider=Google`, using the authorization-code flow with PKCE (public app client, no client secret in the browser). Cognito requires managed login for social IdPs (Cognito docs).
2. Google authenticates the user and returns to Cognito, which creates or links a user in the pool and maps email and name.
3. Cognito redirects back with a code; the SPA exchanges it for an ID token, an access token (1 hour) and a refresh token.
4. The SPA sends the access token as `Authorization: Bearer` on each API call; tokens are kept in memory, not `localStorage`.

How the API validates. Middleware checks the JWT signature against the user pool's JWKS (cached), plus issuer, `token_use=access`, client id and expiry (e.g. the `aws-jwt-verify` library). The `sub` claim becomes the user id; the `cognito:groups` claim gives roles.

Customers vs staff

Who	How assigned	Can do
Customer	Default for any new Google sign-in	Create bookings, upload photos, see **only their own** bookings (`WHERE customer_sub = $sub`)
Staff	An admin adds the user to the Cognito `staff` group (console, CLI or a small admin endpoint), audited in CloudTrail	All of the above, plus list all bookings, view photos, assign technicians

Why editing the frontend does not help an attacker. Hiding admin screens in React is only cosmetic. Every `/api/admin/*` route checks `cognito:groups` contains `staff` and returns 403 otherwise; the token is signed by Cognito, so a user cannot add the group themselves. Group changes take effect when the user's token next refreshes, so token lifetime is kept short.

Optional hardening. Use a Cognito pre-sign-up trigger to block disposable addresses, require staff to use the company Google Workspace domain, and turn on Cognito threat protection (Plus tier) if account takeover becomes a concern. Cognito Essentials and Lite include 10,000 free monthly active users for social sign-in, which covers this app (pricing).

7. Background processing and reliability

SQS delivers each message **at least once**, not exactly once, so the design makes sending idempotent and uses a transactional outbox so a committed booking can never lose its notification job.

Why background processing. SES and SMS calls take hundreds of milliseconds, can be throttled and occasionally fail. Doing them inside the HTTP request would slow every booking and turn a carrier outage into a failed booking.

Creating the job: transactional outbox (justified, and small). Writing to PostgreSQL and SQS are two separate systems; if the API commits the booking and then crashes before `SendMessage`, the job is lost. So the API inserts the booking and an `outbox` row (`booking_id`, `type`, `payload`, `sent_at NULL`) in one transaction, commits, and returns 201. It then tries to publish immediately; a relay loop in the worker service also publishes any row still unsent after 30 seconds, using `SELECT … FOR UPDATE SKIP LOCKED` so two workers never grab the same row. Cost: one table and ~50 lines of code. Without it, an occasional crash means a customer never gets a confirmation.

Consuming jobs

- Worker long-polls (`WaitTimeSeconds=20`, up to 10 messages per call).
- Visibility timeout 60 s — several times the worst-case send time — so a message is not redelivered while still being processed.
- On success the worker deletes the message; on failure it does nothing and SQS redelivers after the timeout.
- Redrive policy `maxReceiveCount = 5` moves a poison message to the DLQ (retention 14 days).
- ECS scales worker tasks on queue depth (`ApproximateNumberOfMessagesVisible`), between 1 and 4.

Duplicates. One message carries one booking; the worker keeps a `notification` table with a unique key on (`booking_id`, `channel`). Before sending it inserts or reads that row; if the channel is already `sent` it skips and deletes the message. Email and SMS are tracked separately, so an SMS retry never re-sends a successful email. The small remaining window (send succeeded, crash before marking `sent`) can cause a rare duplicate message, which is acceptable for a confirmation.

Investigating and retrying failures. The `notification` table stores the last error and attempt count, and staff screens can show "confirmation failed". An alarm fires when the DLQ has any message. After fixing the cause (e.g. a bad phone number or an SMS

template mismatch), an operator uses SQS **DLQ redrive** to move messages back; idempotency makes this safe. SES bounces and complaints, and SMS delivery events, are sent to CloudWatch so delivery failures after the API call are visible too.

8. Monitoring and deployment

Ten alarms cover the failures that actually hurt customers, and every change ships through one GitHub Actions pipeline with automatic rollback.

Logging. API and worker write structured JSON logs (request id, user `sub`, booking id, latency) to CloudWatch Logs via the ECS `awslogs` driver, 30-day retention in prod. CloudTrail records every AWS API call (including who added a user to `staff`) to an S3 bucket. ALB access logs and VPC flow logs are optional, turned on when investigating.

Alarms → SNS → on-call email

Signal	Alarm when	Why
ALB `HTTPCode_Target_5XX_Count`	> 1% of requests for 5 min	Customers are seeing errors
ALB `TargetResponseTime` p95	> 1 s for 10 min	Booking form feels slow
ALB `UnHealthyHostCount`	≥ 1 for 5 min	Tasks failing health checks
ECS service CPU / memory	> 80% for 15 min	Need to scale or fix a leak
RDS `CPUUtilization` / `FreeStorageSpace`	> 80% / < 20%	Database saturation or disk filling
RDS `DatabaseConnections`	> 80% of max	Pool leak; risk of refused connections
SQS `ApproximateAgeOfOldestMessage`	> 5 min	Worker stalled; confirmations delayed
DLQ `ApproximateNumberOfMessagesVisible`	> 0	A notification failed permanently
Outbox rows unsent > 5 min (custom metric)	> 0	SQS publish path broken
SES bounce rate / SMS failure rate	> 5%	Sender reputation or template problem

A CloudWatch dashboard shows these together; AWS Budgets alerts at 80% of the monthly budget.

Deployment pipeline (GitHub Actions)

1. Pull request: lint, unit tests, integration tests against PostgreSQL in a container, `cdk diff` posted to the PR.
2. Merge to `main`: build the API and worker images once, tag with the git SHA, push to ECR (scan on push).
3. Deploy to **staging**: `cdk deploy` (infrastructure), run DB migrations as a one-off ECS task, update ECS services, `s3 sync` the React build and invalidate CloudFront, then smoke tests.
4. Manual approval, then the same SHA is promoted to **production**.

Health checks and rollback. ECS rolling deploys keep 100% healthy capacity and the ALB health check gates new tasks; the ECS deployment circuit breaker with rollback returns the service to the last healthy revision if new tasks keep failing (AWS). Migrations follow expand-then-contract (add columns first, remove old ones a release later) so the previous version still works after a rollback. The frontend rolls back by re-syncing the previous build.

Environments. Separate AWS accounts for dev, staging and prod under AWS Organizations, each its own CDK stage with the same code and different sizes. GitHub authenticates through OIDC to a per-account deploy role. AWS-native CodePipeline would also work; GitHub Actions is chosen because the code already lives there and it needs no extra AWS service to run.

9. Cost considerations and trade-offs

At low traffic the bill is dominated by things that charge per hour whether or not anyone books a repair: NAT gateways, the ALB, Fargate tasks and the RDS instance. Request-based services (SQS, Cognito under 10,000 MAU, S3, SES) cost close to nothing at this scale.

Cost driver	Charges when idle?	Dev / staging	Production	Can defer?
NAT gateway	Yes, hourly + per GB	1	2 (one per AZ)	Second NAT can wait; S3 gateway endpoint cuts data charges

Cost driver	Charges when idle?	Dev / staging	Production	Can defer?
Application Load Balancer	Yes, hourly	1 (or share across envs)	1	No
Fargate tasks	Yes, per vCPU/GB-hour	1 API + 1 worker, 0.25 vCPU; scale to 0 at night	2 API + 1–4 worker, 0.5 vCPU	Fargate Spot for dev/worker
RDS PostgreSQL	Yes, hourly + storage	Single-AZ, burstable (t4g)	Multi-AZ, burstable or m-class	Multi-AZ only in prod
CloudWatch Logs	Per GB ingested/stored	7-day retention	30-day retention, no debug logs	—
WAF	Monthly per web ACL + rules	Off	On	Can start without
SMS	Per message, varies by country/route	Sandbox / test numbers	Real volume	— (SMS is the main per-booking cost)
SES, SQS, S3, Cognito	Usage-based, low	~0	Low	—

I have not put a total on this: prices in ap-south-1 differ from us-east-1 and depend on the instance sizes chosen, so the figure should come from the AWS Pricing Calculator once sizes are agreed. As a reference point, a NAT gateway in us-east-1 is billed per hour plus per GB processed, at $0.045 each (VPC pricing).

Alternative considered: serverless (API Gateway + Lambda + SQS-triggered Lambda + Aurora Serverless v2)

Criterion	ECS Fargate (recommended)	Serverless
Operational complexity	Low; one Docker image per service, same code locally	Low infra, but more functions, packaging and IAM wiring
Cost at very low traffic	Fixed floor (ALB, tasks, NAT)	Near zero compute; DB and NAT still fixed
Scalability	Autoscaling in minutes	Instant, per request
Availability	Multi-AZ tasks behind ALB	Multi-AZ by default

Criterion	ECS Fargate (recommended)	Serverless
Performance	No cold starts; warm DB pools	Cold starts on the Node runtime
PostgreSQL connections	Small, stable pool per task	Needs RDS Proxy to avoid connection storms
Background work	Long-running worker, easy outbox relay	Natural fit (SQS → Lambda); outbox relay needs a schedule
Development	Normal Express app, easy to test	Function-oriented code, local emulation needed

Verdict: containers on Fargate. The exercise describes a classic long-running Node.js API with a PostgreSQL pool and a worker; Fargate runs exactly that with no refactoring, and the fixed cost floor is modest. Serverless becomes attractive only if traffic is extremely spiky or near zero for most of the day.

10. Final recommendation

Build FixFast on ECS Fargate + RDS PostgreSQL + SQS in one Region across two AZs, fronted by CloudFront and secured by Cognito with Google; launch with the essential set in Section 3 and add the deferred items as traffic and risk grow.

Requirements-to-architecture check

Exercise item	Covered by	Status
React website	S3 + CloudFront (Section 3)	Met
Node.js backend API	ECS Fargate api service behind ALB	Met
PostgreSQL	RDS for PostgreSQL, Multi-AZ in prod	Met
Photo storage	Private S3 bucket, presigned uploads	Met
Message queue	SQS + DLQ	Met
Background worker sends SMS + email	ECS worker → SES + End User Messaging SMS	Met
Website never waits	Outbox + async worker (Section 7)	Met
Google sign-in for customers and staff	Cognito + Google IdP	Met
Staff-only admin screens	`staff` group enforced in the API	Met

Exercise item	Covered by	Status
Networking, compute, storage, auth, config/secrets, monitoring, deployment explained	Sections 3–8	Met
Diagram with AWS icons	`fixfast-aws-architecture.drawio`	Met

Open items to confirm

- Customer country and therefore Region; this drives SMS registration (India DLT: entity id, template id, sender id) which can take weeks, so it should start before launch.
- SES production access (leaving the sandbox) and domain verification with DKIM.
- Whether staff accounts must come from a company Google Workspace domain.
- Data retention for photos and personal data.

Appendix A: Architecture decision summary

Component	Selected service	Purpose	Reason	Alternative considered	Status
DNS	Route 53	Domain and alias to CloudFront	Native alias records	External DNS provider	Essential
Frontend hosting	S3 + CloudFront	Serve React build, route `/api/*`	No servers, one domain, edge TLS	Amplify Hosting	Essential
Edge protection	AWS WAF	Managed rules, rate limiting	Stops common attacks at the edge	None (rely on app)	Prod essential
Load balancing	ALB	HTTPS entry to API, health checks	Fits long-running containers	API Gateway	Essential
Compute	ECS on Fargate	Run API and worker containers	No server management, simple model	Lambda; EC2; EKS	Essential
Database	RDS for PostgreSQL	Bookings and users	Managed backups, PITR, Multi-AZ	Aurora PostgreSQL	Essential (Multi-AZ prod)
Photo storage	S3 private bucket	Uploaded images	Cheap, durable, presigned URLs	EFS	Essential
Queue	SQS standard + DLQ	Notification jobs	Managed, retries, redrive	EventBridge; SQS FIFO	Essential

Component	Selected service	Purpose	Reason	Alternative considered	Status
Reliability pattern	Transactional outbox	No lost jobs after commit	Small cost, closes a real gap	Publish after commit only	Essential
Email	Amazon SES	Confirmation email	Native, cheap, delivery events	Third-party email API	Essential
SMS	End User Messaging SMS	Confirmation SMS	Native, India DLT support	SNS SMS; third-party SMS API	Essential
Authentication	Cognito + Google IdP	Google sign-in, JWTs, groups	Managed OIDC, free tier covers app	Auth0; direct Google OAuth	Essential
Config	SSM Parameter Store	Non-secret settings	Free, versioned	Env vars in task definition	Essential
Secrets	Secrets Manager + KMS	DB credentials, rotation	RDS-managed rotation	Parameter Store SecureString	Essential
Monitoring	CloudWatch + SNS	Logs, metrics, alarms, alerts	Native to every service used	Datadog / Grafana	Essential
Audit	CloudTrail	Who changed what	Required for security review	—	Essential
Threat detection	GuardDuty (+ S3 malware scan)	Detect compromise, scan uploads	Low effort to enable	—	Optional
CI/CD	GitHub Actions + ECR	Build, test, deploy	Code already in GitHub	CodePipeline + CodeBuild	Essential
Infrastructure as code	AWS CDK (TypeScript)	Repeatable environments	Same language as the app	Terraform	Essential
Private S3 access	S3 gateway endpoint	Keep S3 traffic off NAT	Free	Interface endpoints	Essential
Second NAT, interface endpoints, RDS Proxy	—	Higher availability, private egress	Add as load and risk grow	—	Optional

Appendix B: 5–10 minute walkthrough

1. The problem (30 s). "FixFast needs a booking site with photo uploads, staff admin screens, Google sign-in, and confirmation SMS and email that never slow the site down. Traffic is small, the team is small, so I optimised for managed services and simplicity."

2. The big picture (1 min). Point at the three boundaries on the diagram. "Left is the public edge: Route 53 for DNS, CloudFront as the single front door, WAF in front of it. In the middle is a private network, the VPC, across two data centres called Availability Zones. On the right are managed services the app calls: queue, email, SMS, secrets, monitoring."

3. Opening the site and signing in (1 min). "CloudFront serves the React files from a private S3 bucket. Sign-in goes to Cognito, which hands off to Google. Google proves who you are; Cognito gives the browser a signed token. The app never stores passwords."

4. Making a booking (2 min). Follow arrows 5–8. "The browser calls `/api/bookings` with that token. CloudFront forwards it to the load balancer, which only accepts traffic from CloudFront. A Node.js container on Fargate checks the token, saves the booking and a 'send confirmation' job row in one PostgreSQL transaction, and immediately replies. The photo goes straight from the browser to S3 using a five-minute signed link, so large files never pass through our servers."

5. Background notifications (1.5 min). Follow arrows 9–12. "The job is published to SQS. A separate worker container picks it up and sends the email through SES and the SMS. If a provider fails, the message comes back and is retried; after five attempts it lands in a dead-letter queue and we get an alarm. SQS can deliver twice, so the worker records what it already sent and skips duplicates. The outbox row means that even if we crash right after saving, the job is not lost."

6. Security (1 min). "Only the load balancer is public. Containers and the database live in private subnets; security groups allow load balancer → API → database and nothing else. Staff screens are protected on the server: the API checks the token's `staff` group, so editing the React code achieves nothing. Credentials live in Secrets Manager and rotate automatically; everything is encrypted."

7. Operations (1 min). "Logs and metrics go to CloudWatch, with about ten alarms on things customers feel: errors, slow responses, a stuck queue, failed notifications. Code ships from GitHub Actions; the infrastructure is AWS CDK code; ECS rolls back automatically if a new version fails health checks; dev, staging and prod are separate accounts."

8. Trade-offs and close (1 min). "I compared this with a Lambda-based design. Lambda is cheaper when idle but needs a connection proxy for PostgreSQL and reshapes the code; Fargate runs the app as written. The main fixed costs are NAT gateways, the load balancer and the database, so dev runs smaller and production adds Multi-AZ and a second NAT. One thing to start early: SMS sender registration in India can take weeks."

Sources

- Cognito: social identity providers

- Cognito pricing
- End User Messaging SMS: India sender ID registration
- ECS deployment circuit breaker
- Amazon VPC pricing
- NAT gateway pricing guidance